Spiff v2 — build contract for view modules

Read this fully before writing a line. Everything here is already implemented and loaded before your file runs. Your job is to write one file and nothing else.

---

1. What you are building

A clickable, made-up HTML mockup. No architecture, no LLM, no live data, no backend. Every number is invented but must be plausible and internally consistent. This is the interior of the car, not the engine.

The product: Spiff — a self-service AI reporting agent for UBT's GST division (a religious community's operations arm in South Africa). A person asks a question in plain language; an agent interrogates UBT systems (Directory, Orbit, Assemble, the warehouse) and returns a governed, shareable answer.

Three governance invariants that must be visible in whatever you build:
1. Everything runs as its owner and re-checks identity and permissions on every run.
2. Shared answers re-run per viewer, scoped to that person — sharing organises, it never widens access.
3. Sharing an automation clones it; each copy runs as its owner.

House voice: confident present tense, plain English, no jargon, no marketing fluff, no emoji. Write microcopy the way a good product writes it — short, specific, human. Say what a thing is, not what it "enables". Where a screen makes a governance promise, state the promise in the UI.

---

2. File format — follow exactly

Write a single file at the path you are given, e.g. src/views/62-catalog.js.

```
<script>
/ ---------- Data catalogue ---------- /
const CATALOG_STATE = { q:"", domain:"all" }; // module-local state, uniquely named

function renderCatalog(){
$('#view-catalog').innerHTML = '...';
// wire events after innerHTML
}
V2ROUTES.catalog = renderCatalog; // register the route
</script>
```

Rules:
- The file must open with <script> and close with </script> on their own lines.
- It is concatenated into one page, so every top-level name must be globally unique. Prefix module-local names with your screen (CATALOG_, renderCatalog, catalogRow, …).
- Never redeclare anything listed in §4. const collisions are a hard page crash.
- Plain ES2019 in a classic <script>: no modules, no imports, no optional chaining needed but fine, no top-level await.
- Build strings with '...'+x+'...' concatenation (the codebase style) — template literals are fine too but keep them out of onclick="" attributes.
- Inline handlers call global functions: onclick="openDataset('meetings')". Any function referenced from HTML must be a top-level function declaration.
- Escape all user-ish strings with esc(...).
- No localStorage beyond what already exists. No network calls. No external assets. Any image must be an inline SVG.

---

3. Design system — use these, do not invent new ones

Fonts are already loaded: Schibsted Grotesk (headings, var(--dsp)), IBM Plex Sans (body), IBM Plex Mono (.mono, numbers). Palette is UBT navy/blue. Light and dark both matter — only ever use CSS variables for colour, never a literal hex, except for the seeded avatar colours returned by avColor().

Tokens: --ground --surface --card --ink --muted --hair --hair2 --navy --blue --accent --accent-soft --good --good-soft --warn --warn-soft --crit --crit-soft --ok --ok-soft --info --info-soft --purple --teal --pink --measure --attr --filt --shadow --shadow-lg --r --r-sm --r-lg

Layout

.view.wide is your section (max 1340px, already applied).
.v2head page header · .split (main + 320 side) · .split.left (280 side + main) · .split.wide · .stack · .rowflex · .sp (flex spacer) · .g2 .g3 .g4 grids · .hairline · .stickytop · .scrollx

Surfaces

.panel + .panel-h / .panel-b / .panel-b.tight / .panel-f — use the panel() helper.
.kpi (use kpi()), .callout.info|ok|warn|crit|mut (use callout()), .defblock, .codeblock.

Controls (from v1, still current)

.btn, .btn.pri, .btn.sm, .btn.danger, .btn.ghost · .seg/.seg2 segmented · .field + label + select · .tabs (use tabsHTML() + pane()) · .sw switch (use sw()) · .tri three-state permission (use tri()) · .fchip2 filter chip · .bigsearch · .facets/.facet

Data display

.dtbl + .dtbl-wrap tables (th.num, td.num, .tech) · .lrow list rows · .kv / .kvlist key-value · .meter · .ring · .spark · .matrix + .cell.full|part|none|block · .tline + .tev timeline · .steps wizard · .tok.m|.a|.f query tokens · .bdg badges · .av2 / .avstack / .person avatars · .empty2 empty state · .sentence + .slot rule builder · .pickcard selectable card.

Charts: renderChart(containerEl, chartObj, type) exists (ECharts, inlined) — only use it if you genuinely need a chart; sparkline(arr) and .meter cover most cases and are cheaper.

---

4. Globals available to you — DO NOT REDECLARE

From the v1 engine

$(sel) querySelector · esc(s) · toast(msg) · go(view, arg) · showOnly(id) · navActive(v) · closeModal() · openFromCard(answerId) · askText(text) · ICON (v1 icon set) · ANSWERS CHIPS THEME REPORTS CHATS PEOPLE_NAMES→PEOPLE (see below) TEAMS LIBRARY
CAPS CAPCFG DASHBOARDS TRIGTYPES CADENCES SOURCES METRICS OPS WORKFLOWS SUBS
SCHEDRUNS TRIGRULES · renderChart repaintAllCharts groupsHTML collapseBtn toggleGroups · saveReport shareModal scheduleModal subscribeModal
NOTE: v1 has a const PEOPLE = ["Innocent B.", …] array of names. v2 renames nothing — the v2 people objects live in PEOPLE in data/50-org.js which is loaded after v1 and

shadows it. Use the v2 PEOPLE (objects). If you need the old name list, use PEOPLE.map(p=>p.name).

From data/50-org.js

- ORG — {name, short, users:428, activeThisWeek, regions[9], divisions[], tenant}
- ME — the current user (Innocent Bhengu, LDM Coordinator, 4 of 6 regions)
- SYSTEMS[] — {id,name,kind,desc,status,latency,datasets,color}; sysById(id)
- ROLES[] — {id,name,members,desc,privs[],risk}; roleById(id)
- GROUPS[] — {id,name,members,type,roles[],owner,rule?}; groupById(id)
- PEOPLE[] — 32 people
{id,name,initials,title,team,region,groups[],roles[],status,last,asked,flag?,me?}
plus PEOPLE_HIDDEN (396 more not listed). personByName(n)
- DATASETS[] — 15 datasets, the heart of the mockup. Each:
{id,name,tech,sys,domain,sens,cert,certBy,certOn,warning?,purpose,grain,rows,rowTrend[12],
refreshed,next,sla,freshness,quality:{score,completeness,validity,freshness,uniqueness},
incidents,coverage,exclusions,population,owner,steward,sme,channel,popularity,rank,users,
questions[],joins[],access,accessNote,rules[],fields[]}
each field: {label,tech,type:'attribute'|'measure'|'date'|'geo',desc,example,sens,mask,usage,
complete,distinct,format,synonyms[],glossary}
Helpers: ds(id), DOMAINS
- GLOSSARY[] — {term,def,owner,version,agreed,used}
- CERT / SENS / FRESH lookup maps · avColor(name) · initialsOf(name) · F(...) field factory

From core/45-ui2.js

I2 (icon set — see the file for names) · avatar(name,size?) · personChip(name,sub) ·
bdg(text,cls,icon) · certBadge(dataset) · sensBadge(sens) · freshBadge(freshness) ·
meter(pct,cls) · ring(pct,cls) · scoreCls(n) · kpi(label,value,sub,delta) ·
sparkline(arr) · panel(title,body,{icon,sub,act,foot,tight,cls,id}) ·
callout(kind,html,icon) · pageHead({eyebrow,title,desc,badges,acts,back}) ·
tabsHTML(group,[[id,label,count?],…],active) + pane(group,id,html,on) + switchTab(g,t) ·
sw(on,onclick) · tri(value) · setTri(btn) · emptyState(title,sub,icon) ·
openModal(html,widthPx?) + modalFoot(cancelLabel,confirmLabel,onConfirmJs) ·
fmt(n) · cls(...) · esc2(s) ·
SIM / startSim(personName) / stopSim() / viewer() — the viewer simulator ·
V2ROUTES · CRUMB2 · refreshView() · crumbTrail([[label,jsOrNull],…])

Other v2 data modules (load before views; assume they exist)

- data/51-rules.js → RULES[], ruleById(id), RULE_CATEGORIES, MASK_METHODS,
HIT_POLICIES
- data/52-access.js → REQUESTS[], REVIEWS[], ENTITLEMENTS (group × dataset)
- data/53-mcp.js → CONNECTORS[], SPIFF_TOOLS[], MCP_LOG[]
- data/54-audit.js → AUDIT[], AUDIT_KINDS
- data/55-chat.js → THREADS[], SKILLS[]

---

5. Cross-screen navigation — use these names

Already routed or to be provided by sibling modules. Call them freely; if one is missing at runtime the router shows a graceful error rather than crashing the page.

Function	Goes to
go('home')	Home

go('ask') / openFromCard(id)	Ask / an answer
go('chat') / openThread(id)	Portal chat
go('catalog')	Data catalogue
openDataset(id, tab?)	Dataset profile
go('rules') / openRule(id)	Business rules
go('people') / openPerson(name)	People & access
go('myaccess')	My data access
go('mcp') / openConnector(id)	Connectors
go('audit')	Activity log
go('glossary')	Definitions
requestAccessModal(datasetId)	Request-access modal
startSim(personName) / stopSim()	Viewer simulator
startOnboarding()	First-run overlay

---

6. Quality bar

- Density with air. These are working screens for people who do this all day. Show real counts, real names, real timestamps. But keep whitespace — never a wall of grey.
- Every screen answers "so what?" A page head with an eyebrow, a title and one sentence of plain-English purpose. Not "Manage your datasets" — say what the screen is for.
- No lorem ipsum, no TODO, no placeholder. Every string ships.
- Interactive, not static. Filters filter. Tabs switch. Rows open. Toggles toggle and toast(). Modals open and close. At minimum: search/filter works, and every row leads somewhere.
- Governance is visible. Wherever data is shown, say whose scope it is under. Wherever something is hidden, say so and why — "no data" and "not allowed to see the data" must look different.
- Dark mode works. Only CSS variables for colour.
- Responsive-ish. Grids collapse (the .g2/.g3/.g4 and .split classes already do).
- Aim for 250–450 lines. Rich, but readable.

---

7. How to check your work

You cannot run a browser. Instead:
1. cd /home/claude/spiff/v2 && node -e "new Function(require('fs').readFileSync('src/views/YOURFILE.js','utf8').replace(/<\/?script>/g,''))" — catches syntax errors.
2. Re-read your file for: unbalanced quotes inside onclick="", missing esc(), redeclared globals, any hard-coded hex colour, any reference to a helper not in §4.
3. Confirm every onclick target is a function you defined or one listed in §5.